



2



6



720 sp

عملي

كلية الهندسة المعلوماتية

السنة الثالثة

العودية و تمثيل المعرفة

عملي مشترك



RB Informatics; 19/04/2026

محتوى مجاني غير مخصص للبيع التجاري

مبادئ الذكاء الصناعي

تحدثنا في المحاضرة السابقة عن مفاهيم البرمجة التصريحية (Declarative) والأمرية (Imperative)، وسنبداً بتطبيق عملي على لغة Prolog من خلال مثال شجرة العائلة:

علاقة الأبوة: (Parent) تربط بين الآباء والأبناء مثل: `parent(ahmad, adnan)`.

ومن خلال هذه الحقائق والعلاقات سنوجد القواعد التالية:

قاعدة الأب (Father): `father(X, Y) :- male(X), parent(X, Y)`.

يكون X أباً لـ Y إذا كان ذكراً وله علاقة أبوة معه.

قاعدة الزوج (Husband): `husband(X, Y) :- male(X), female(Y), parent(X, Z), parent(Y, Z)`.

الزوج ذكر، والزوجة أنثى، ويشتركان في كونهما أبوين لنفس الشخص (Z).

قاعدة الجد (Grandfather): `grandfather(X, Y) :- male(X), parent(X, Z), parent(Z, Y)`.

علاقة (Grandfather): يكون الشخص (X) جداً لـ (Y) إذا كان (X) ذكراً وأباً لشخص (Z)، وهذا الشخص (Z) هو أب لـ (Y).

مفهوم السلف (Ancestor) باستخدام العودية (Recursion):

تعرف العودية في المنطق والبرمجة بأنها عملية حل مشكلة عن طريق تقسيمها إلى مشكلات أصغر من نفس النوع. في حالة تعريف السلف، لا نحتاج لكتابة قواعد منفصلة لكل جيل، بل نستخدم قاعدة واحدة تستدعي نفسها لتتبع شجرة العائلة.

القاعدة تقول إن الشخص يكون أصلاً لآخر في حالتين:

• شرط التوقف: `ancestor(X, Y) :- parent(X, Y)`.

يكون أصلاً إذا كان الأب المباشر

• الاستدعاء العودي: `ancestor(X, Y) :- parent(Z, Y), ancestor(X, Z)`.

يكون X أصلاً لـ Y إذا كان هناك Z هو أب لـ Y ، وكان X أصلاً لهذا الأب Z



آلية عمل محرك الاستنتاج (Inference Engine):

يمر المحرك على القواعد بالترتيب؛ يبدأ بالبحث عن الأب المباشر، ثم ينتقل للبحث عودياً في شجرة العائلة صعوداً. المحرك لا يستخدم كلمة "Return" كما في اللغات الأخرى، بل يقوم بتعبئة المتحولات (Variables) بالنتائج التي تحقق القواعد.

عند إرسال استفسار (Query)، قد يعطينا البرنامج نتيجة واحدة، وعند طلب (Next) يبحث المحرك عن قيم أخرى تحقق نفس القاعدة، وهكذا حتى يستنفد جميع الاحتمالات.

✚ تطبيق قاعدة الزوج (Husband):

`husband(X, Y) :- male(X), female(Y), parent(X, Z), parent(Y, Z).`

إذا سألنا: `husband(adnan,yara)`

يتحقق المحرك:

- هل عدنان ذكر؟ (نعم) هل يارا أنثى؟ (نعم)
- يبحث عن ابن مشترك (Z) يجد أن عدنان والد لـ أحمد Z = أحمد
- يتحقق: هل يارا هي أيضاً والدة لـ أحمد؟ (نعم)
- النتيجة: true.

✚ مثال السلف (Ancestor) بالعودية:

`ancestor(X, Y) :- parent(Z, Y), ancestor(X, Z).`

`parent(grandfather,adnan)`

خطوات التنفيذ:

- شرط التوقف: هل الجد هو الأب المباشر لـ احمد؟ لا.
- الاستدعاء العودي: يبحث عن والد احمد (يجد أنه عدنان).
- يتحول السؤال إلى: هل الجد سلف لـ عدنان؟
- يتحقق من شرط التوقف: هل `parent(grandfather,adnan)` موجودة؟ نعم.
- النتيجة: true.

✚ تعبئة المتحولات (Variables):

هل `Parent(yara,Children)`

○ النتيجة: `Children = ahmad`

وعند طلب (Next) إن لم توجد حقائق أخرى يتوقف المحرك لأنه استنفد جميع الاحتمالات.

أنواع البيانات والعمليات الرياضية في Prolog:

- الذرة (Atom): هي أي ثابت يبدأ بحرف صغير (مثل ahmad)
- الأرقام (Numbers): الأعداد العادية
- البيانات المعقدة (Complex Data Types): مثل تعريف "الخط" (Line) بداخله "نقطتان" (Points)، وكل نقطة لها إحداثيات (X, Y)

`line(point(X, Y), point(X2, Y2)).`

- الخط العمودي (Vertical Line): لمعرفة ما إذا كان الخط عمودياً بناءً على ثبات قيمة X:

`vertical(line(point(X, Y), point(X, Z))).`

- العمليات الحسابية: تدعم Prolog الجمع (+) والطرح (-) والضرب والقسمة لكنها لا تستخدم للإسناد المباشر

عندما تريد استخدام Prolog لحساب قيمة مثل $Y = X + 3$ ، لا تقوم بكتابة عملية حسابية مباشرة كما تفعل في لغات البرمجة التقليدية (مثل Python أو C). بدلاً من ذلك، Prolog يتعامل مع ذلك من خلال البحث عن القيم المناسبة التي تحقق الشروط المحددة.

لنفترض أننا نريد معرفة قيمة Y عندما يكون $X = 4$. في Prolog، يمكنك كتابة قاعدة مثل:

`sum(X, Y) :- Y is X + 3.`

عندما تستفسر عن `sum(4, Y)` يقوم Prolog بالبحث في قاعدة البيانات عن القيم التي تحقق هذه القاعدة. هنا، Prolog سيقوم بتنفيذ العملية $Y is 4 + 3$ لكن الفكرة هي أنه يبحث في القاعدة ليجد القيم التي تحقق العلاقة ومن الممكن أن يكون ل Y أكثر من قيمة.

- المساواة وعدم المساواة: تستخدم `=` للتعبير عن لا يساوي مثال:

`sibling(X, Y) :- parent(Z, X), parent(Z, Y), X \= Y.`

هنا استخدمناها للتأكد من أن الشخص ليس أماً لنفسه في علاقة الـ Sibling

ملاحظات:

is تأخذ الجانب الأيمن كتعليمات حسابية وتقوم بتقييمه ويجب أن يكون الجانب الأيسر متغيراً ولا يمكن استخدامها لتعيين قيمة لمتغير بناءً على قيمة أخرى مباشرة (مثل $X is Y + 3$ إذا كان Y غير معروف)

إذا أردنا كتابة اسم يبدأ بحرف كبير (مثل اسم الشخص) ك ثابت (Atom) وليس كمتحول يجب وضعه بين علامتي تنصيص مثل 'Ahmed' وإلا سيعتبره البرنامج متحولاً

في حال أضاف شخص ما حقيقة (Fact) خاطئة أو "غبية" (مثل أن الشخص هو أب لنفسه)، فإن برنامج "السلف" (Ancestor) سيدخل في حلقة لانهاية ولن يتوقف عن البحث أبداً

٤ تطبيق العودية الرياضية (سلسلة فيبوناتشي):

هي سلسلة من الأعداد حيث يكون كل عدد هو مجموع العددين السابقين له، وتبدأ السلسلة عادة بالأعداد 0 و 1. لذلك، فإن الأعداد في سلسلة فيبوناتشي تكون كالتالي: 0، 1، 1، 2، 3، 5، 8، 13، وهكذا.

$\text{fibonacci}(0, 0).$

$\text{fibonacci}(1, 1).$

$\text{fibonacci}(Z, F) :- Z > 1, Z1 \text{ is } Z - 1, Z2 \text{ is } Z - 2, \text{fibonacci}(Z1, F1), \text{fibonacci}(Z2, F2), F \text{ is } F1 + F2.$

أولاً: تعريف حالات التوقف (Base Cases)

في البرمجة العودية يجب تحديد نقاط توقف ثابتة لكي لا يستمر البرنامج في الاستدعاء الذاتي إلى ما لا نهاية أي نعرف

- الحقيقة الأولى: قيمة العدد في الموقع (0) هي (0)، وتكتب برمجياً.. $\text{fibonacci}(0, 0)$;
- الحقيقة الثانية: قيمة العدد في الموقع (1) هي (1)، وتكتب برمجياً.. $\text{fibonacci}(1, 1)$;

ثانياً: القاعدة العودية (Recursive Rule)

لحساب أي قيمة في موقع أكبر من (1) نستخدم القاعدة التي تنص على أن العدد هو مجموع العددين السابقين له وتتم هذه العملية عبر الخطوات التالية الموضحة في الكود:

- شرط التحقق: نضع الشرط $Z > 1$ لضمان أن العملية الحسابية لن تستمر إلى أرقام سالبة ولتوجيه البرنامج نحو حالات التوقف.

○ يتم حساب الموقع السابق مباشرة $Z1 \text{ is } Z - 1$

○ يتم حساب الموقع الذي يسبقه $Z2 \text{ is } Z - 2$

- الاستدعاء الذاتي: (Recursion)

○ يقوم البرنامج باستدعاء الدالة لنفسها لإيجاد قيمة الموقع $Z1$ وتخزين النتيجة في المتغير $F1$

○ ثم نستدعيها مرة أخرى لإيجاد قيمة $Z2$ وتخزين النتيجة في المتغير $F2$

- في الخطوة الأخيرة يتم جمع القيمتين المستخرجتين $(F \text{ is } F1 + F2)$ للحصول على القيمة النهائية المطلوبة للموقع Z

تمثيل المعرفة (Knowledge Representation) باستخدام منطق الرتبة الأولى: (First-Order Logic)

وهو أسلوب يهدف إلى تحويل الجمل البشرية والواقع المحيط بنا إلى صيغ رياضية وبرمجية دقيقة. الهدف الأساسي من هذا التحويل هو بناء قاعدة معرفة (Knowledge Base - KB) تمكن الحاسوب من "التفكير" والاستنتاج بطريقة منطقية مشابهة للعقل البشري، مما يسمح للأنظمة الذكية باستخراج حقائق جديدة لم تذكر لها صراحة.

٤ مراجعة لبعض الرموز:

- مكمم الشمول (Universal Quantifier):
الاستخدام: نستخدمه عندما تنطبق الصفة على الجميع بلا استثناء (كل، جميع، أي).
المعنى: "لأي عنصر x إذا تحقق الشرط الأول، فإنه بالضرورة يحقق الشرط الثاني."
 - مكمم الوجود (Existential Quantifier):
الاستخدام: نستخدمه عندما نتحدث عن عنصر واحد على الأقل (يوجد، بعض، واحد على الأقل).
المعنى: "يوجد عنصر x يحقق الصفة الأولى و يحقق الصفة الثانية في نفس الوقت"
 - أدوات الربط المنطقي (Logical Connectives)
هذه الرموز هي التي تربط بين الحقائق لبناء جمل معقدة:
 - التقاطع (Conjunction):
تستخدم لربط شرطين يجب أن يتحققا معاً لتكون الجملة صحيحة.
 - يقتضي (Implication):
تستخدم لتمثيل القواعد والشروط (إذا حدث كذا، سيحدث كذا).
 - الاجتماع:
- تعني أن الجملة تكون صحيحة إذا تحقق الشرط الأول، أو الشرط الثاني، أو كلاهما معاً.

أمثلة على تحويل العبارات اللغوية إلى رموز منطقية:

- "All crows are black" (كل الغربان سوداء)
 - الصيغة: $\forall x, C(x) \Rightarrow B(x)$
 - تعني أنه لأي عنصر (x) في الكون، إذا كان هذا العنصر غراباً (C) ، فإنه بالضرورة يكون أسود (B) . هذا يسمى التعميم الكلي
- Every gardener likes the sun كل بستاني يحب الشمس
 - الصيغة: $\forall x, g(x) \Rightarrow \text{Likes}(x, \text{Sun})$
 - توضح أن صفة "حب الشمس" ملازمة لكل من يحمل صفة بستاني
- "John likes to eat everything" جون يحب أن يأكل كل شيء
 - الصيغة: $\forall x, \text{food}(x) \Rightarrow \text{likes}(\text{John}, x)$
 - أي شيء يُصنف كطعام فإن جون يحبه



• "John likes at least one dish Jane likes" (جون يحب طبقاً واحداً على الأقل مما تحبه جين)

○ الصيغة: $\exists x, \text{food}(x) \wedge \text{like}(\text{Jane}, x) \wedge \text{like}(\text{John}, x)$

○ هنا استخدمنا رمز الوجود $\exists$ (ليدل على وجود عنصر واحد على الأقل)

○ (X) هو طعام وتحبه جين ويحبه جون في آن واحد.

كيف يتم ربط الأحداث ببعضها لاستنتاج نتائج:

• "If it is hot and humid, then it will be raining" (إذا كان الجو حاراً ورطباً، فسوف تمطر)

○ بفرض أن H حار، D رطب، R مطر

○ $H \wedge D \Rightarrow R$

• مثال الحرارة والرطوبة "If it is humid, then it is hot": (إذا كان رطباً، فهو حار)

○ الصيغة: $D \Rightarrow H$

○ الحقيقة المعطاة "It is humid": (الجو رطب)

الفائدة: عند إدخال هذه الحقائق في قاعدة المعرفة (KB)، يستطيع النظام استنتاج أن الجو "حار" تلقائياً، ومن ثم استنتاج أنها "سوف تمطر" بناءً على القواعد السابقة



وإلى هنا تكون قد انتهت محاضرتنا
نلتقاكم في المحاضرة القادمة...

انتهت المحاضرة

<The end>